Operators

Search operators

Scope:

All

Unlock more complex operators at Expert, Master and Grandmaster Genius levels.

Arithmetic

Operator	Scope	Description
abs(x) base	Combo, Regular, Selection	Returns the absolute value of a number, removing any negative sign. Show more
add(x, y, filter = false), x + y base	Combo, Regular, Selection	Adds two or more inputs element wise. Set filter=true to treat NaNs as 0 before summing. Show more
densify(x) base	Combo, Regular	Converts a grouping field of many buckets into lesser number of only available buckets so as to make working with grouping fields computationally efficient Show more
divide(x, y), x / y base	Combo, Regular, Selection	x / y
inverse(x) base	Combo, Regular, Selection	1 / x
log(x) base	Combo, Regular, Selection	Calculates the natural logarithm of the input value. Commonly used to transform data that has positive values. Show more
max(x, y, ..) base	Combo, Regular, Selection	Maximum value of all inputs. At least 2 inputs are required Show more
min(x, y ..)	Combo, Regular,	Minimum value of all inputs. At least 2 inputs are required

multiply(x ,y, ... , filter=false), x * y base	Combo, Regular, Selection	Multiplies two or more inputs element wise. Set filter=true to treat NaNs as 0 before multiplication Show more
power(x, y) base	Combo, Regular, Selection	x ^ y Show more
reverse(x) base	Combo, Regular, Selection	- x
s_log_1p(x) genius	Combo, Regular, Selection	Confine function to a shorter range using logarithm such that higher input remains higher and negative input remains negative as an output of resulting function and -1 or 1 is an asymptotic value Show more
sign(x) base	Combo, Regular, Selection	Returns the sign of a number: +1 for positive, -1 for negative, and 0 for zero. If the input is NaN, returns NaN. Input: Value of 7 instruments at day t: (2, -3, 5, 6, 3, NaN, -10) Output: (1, -1, 1, 1, 1, NaN, -1) Show more
signed_power(x, y) base	Combo, Regular, Selection	x raised to the power of y such that final result preserves sign of x Show more
sqrt(x) base	Combo, Regular, Selection	Returns the non negative square root of x. Equivalent to power(x, 0.5); for signed roots use signed_power(x, 0.5). Show more
subtract(x, y, filter=false), x - y base	Combo, Regular, Selection	Subtracts inputs left to right: x ? y ? ... Supports two or more inputs. Set filter=true to treat NaNs as 0 before subtraction. Show more

Logical

Operator	Scope	Description
and(input1, input2) base	Combo, Regular, Selection	Returns 1 ('true') if both inputs are 1 ('true'). Otherwise, returns 0 ('false').
if_else(input1, input2, input 3) base	Combo, Regular, Selection	The if_else operator returns one of two values based on a condition. If the condition is true, it returns the first value; if false, it returns the second value.

input1 < input2 base	Combo, Regular, Selection	Returns 1 ('true') if input1 is a smaller than input2. Otherwise, returns 0 ('false').
input1 <= input2 base	Combo, Regular, Selection	Returns 1 ('true') if input1 is a smaller or the same as input2. Otherwise, returns 0 ('false').
input1 == input2 base	Combo, Regular, Selection	Returns 1 ('true') if input1 and input2 are the same. Otherwise, returns 0 ('false').
input1 > input2 base	Combo, Regular, Selection	Returns 1 ('true') if input1 is a larger than input2. Otherwise, returns 0 ('false').
input1 >= input2 base	Combo, Regular, Selection	Returns 1 ('true') if input1 is a larger or the same as input2. Otherwise, returns 0 ('false').
input1!= input2 base	Combo, Regular, Selection	Returns 1 ('true') if input1 and input2 are different numbers. Otherwise, returns 0 ('false').
is_nan(input) base	Combo, Regular, Selection	If (input == NaN) return 1 else return 0 Show more
not(x) base	Combo, Regular, Selection	Returns the logical negation of x. Returns 0 when x is 1 (‘true’) and 1 when x is 0 (‘false’).
or(input1, input2) base	Combo, Regular, Selection	Returns 1 if either input is true (either input1 or input2 has a value of 1), otherwise it returns 0.

Time Series

Operator	Scope	Description
days_from_last_change(x) base	Combo, Regular	Calculates the number of days since the last change in the value of a given variable. Show more
hump(x, hump = 0.01) base	Combo, Regular	Limits amount and magnitude of changes in input (thus reducing turnover) Show more
inst_tvr(x, d) genius	Combo, Regular	Total trading value / Total holding value in the past d days Input: Value of 1 instrument in past 5 days where first element is the latest: (105, 102, 99, 101,100) Output: 0.022 from (1+2+3+3)/(105+102+99+101)

k, ignore="NaN") base		number of (d) days, with the option to ignore certain values. Commonly used for backfilling missing data. Show more
last_diff_value(x, d) base	Combo, Regular	Returns the most recent value of x from the past d days that is different from the current value of x. Show more
ts_arg_max(x, d) base	Combo, Regular	Returns the number of days since the maximum value occurred in the last d days of a time series. If today's value is the maximum, returns 0; if it was yesterday, returns 1, and so on. Show more
ts_arg_min(x, d) base	Combo, Regular	Returns the number of days since the minimum value occurred in a time series over the past d days. If today's value is the minimum, returns 0; if it was yesterday, returns 1, and so on. Show more
ts_av_diff(x, d) base	Combo, Regular	Calculates the difference between a value and its mean over a specified period, ignoring NaN values in the mean calculation. In short, it returns x – ts_mean(x, d) with NaNs ignored. Show more
ts_backfill(x,lookback = d, k=1) base	Combo, Regular	Replaces missing (NaN) values in a time series with the most recent valid value from a specified lookback window, improving data coverage and reducing risk from missing data. Show more
ts_corr(x, y, d) base	Combo, Regular	Calculates the Pearson correlation between two variables, x and y, over the past d days, showing how closely they move together. Show more
ts_count_nans(x,d) base	Combo, Regular	Counts the number of missing (NaN) values in a data series over a specified number of days. Show more
ts_covariance(y, x, d) base	Combo, Regular	Calculates the covariance between two time-series variables, y and x, over the past d days. Useful for measuring how two variables move together within a specified historical window. Show more
ts_decay_exp_window(x, d, factor = f) genius	Combo, Regular	Returns exponential decay of x with smoothing factor for the past d days Show more

d, dense = false) base		smoothing the data by averaging recent values and reducing the impact of older or missing data. Show more
ts_delay(x, d) base	Combo, Regular	Returns the value of a variable x from d days ago. Use this operator to access historical data points by specifying the desired time lag in days. Show more
ts_delta(x, d) base	Combo, Regular	Calculates the difference between a value and its delayed version over a specified period. Useful for measuring changes or momentum in time-series data. Show more
ts_delta_limit(x, y, limit_volume=0.1) genius	Combo, Regular	Limit the change in the Alpha position x between dates to a specified fraction of y. The "limit_volume" can be in the range of 0 to 1. Also, please be aware of the scaling for x and y. Besides setting y as adv20 or volume related data, you can also set y as a constant.
ts_mean(x, d) base	Combo, Regular	Calculates the simple average (mean) value of a variable x over the past d days. Show more
ts_median(x, d) genius	Combo, Regular	Returns median value of x for the past d days
ts_product(x, d) base	Combo, Regular	Returns the product of the values of x over the past d days. Useful for calculating geometric means and compounding returns or growth rates. Show more
ts_quantile(x,d, driver="gaussian") base	Combo, Regular	Calculates the ts_rank of the input and transforms it using the inverse cumulative distribution function (quantile function) of a specified probability distribution (default: Gaussian/normal). This helps to normalize or reshape the distribution of your data over a rolling window. Show more
ts_rank(x, d, constant = 0) base	Combo, Regular	Ranks the value of a variable for each instrument over a specified number of past days, returning the rank of the current value (optionally adjusted by a constant). Useful for normalizing time-series data and highlighting relative performance over time. Show more
ts_regression(y, x, d, lag = 0, rettype = 0) base	Combo, Regular	Returns various parameters related to regression function Show more
ts_scale(x, d, constant = 0)	Combo, Regular	Scales a time series to a 0–1 range based on its minimum and maximum values over a specified period, with an optional constant shift.

ts_std_dev(x, d) base	Combo, Regular	Calculates the standard deviation of a data series x over the past d days, measuring how much the values deviate from their mean during that period. Show more
ts_step(1) base	Combo, Regular	Returns a counter of days, incrementing by one each day. Show more
ts_sum(x, d) base	Combo, Regular	Sum values of x for the past d days.
ts_target_tvr_decay(x, lambda_min=0, lambda_max=1, target_tvr=0.1) genius	Combo, Regular	Tune "ts_decay" to have a turnover equal to a certain target, with optimization weight range between lambda_min, lambda_max
ts_zscore(x, d) base	Combo, Regular	Calculates the Z-score of a time series, showing how far today's value is from the recent average, measured in standard deviations. Useful for standardizing and comparing values over time. Show more

Cross Sectional

Operator	Scope	Description
normalize(x, useStd = false, limit = 0.0) base	Combo, Regular	Centers a daily cross section by subtracting the market mean; optionally divide by the cross sectional standard deviation and clamp the result to [?limit, +limit]. NaNs are ignored in mean/std. Show more
quantile(x, driver = gaussian, sigma = 1.0) base	Combo, Regular	Ranks and shifts a vector of Alpha values, then applies a chosen statistical distribution (gaussian, cauchy, or uniform) to reduce outliers. The sigma parameter controls the scale of the output. Show more
rank(x, rate=2) base	Combo, Regular	Ranks the values of the input x among all instruments, returning numbers evenly spaced between 0.0 and 1.0. Useful for normalizing data and reducing the impact of outliers. Show more
regression_neut(y, x) genius	Combo, Regular	Conducts the cross-sectional regression on the stocks with Y as target and X as the independent variable Show more

longscale=1, shortscale=1) base		equals a specified book size. Allows separate scaling for long and short positions using optional parameters. Show more
truncate(x,maxPercent=0.01) genius	Combo, Regular	Operator truncates all values of x to maxPercent. Here, maxPercent is in decimal notation Show more
winsorize(x, std=4) base	Combo, Regular	Winsorize limits values in a data to within a specified number of standard deviations from the mean, reducing the impact of extreme outliers. Show more
zscore(x) base	Combo, Regular	Z-score is a numerical measurement that describes a value's relationship to the mean of a group of values. Z-score is measured in terms of standard deviations from the mean Show more

Vector

Operator	Scope	Description
vec_avg(x) base	Combo, Regular	Calculates the mean (average) of all elements in a vector field for each instrument and date, converting vector data to a single matrix value. Show more
vec_count(x) genius	Combo, Regular	Number of elements in vector field x
vec_max(x) genius	Combo, Regular	Maximum value form vector field x
vec_min(x) genius	Combo, Regular	Minimum value form vector field x
vec_range(x) genius	Combo, Regular	Difference between maximum and minimum element in vector field x
vec_stddev(x) genius	Combo, Regular	Standard Deviation of vector field x
vec_sum(x) base	Combo, Regular	Calculates the sum of all values in a vector field. Show more

Transformational

bucket(rank(x), range="0, 1, 0.1", skipBoth=False, NaNGroup=False) or bucket(rank(x), buckets = "2,5,6,7,10", skipBoth=False, NaNGroup=False) base	Combo, Regular	The bucket operator creates custom groups by dividing data into buckets (ranges) based on ranked values of any data field. These buckets can then be used with group operators like group_neutralize, group_rank, group_zscore etc. Show more
generate_stats(alpha) base	Combo	The generate_stats() operator calculates Alpha statistics for each day in the IS period. It takes an input of selected Alphas with shape = (A x D x I). It outputs daily statistics for each Alpha with shape = (S x D x A), where S is the number of statistics calculated.
left_tail(x, maximum = 0) genius	Combo, Regular	NaN everything greater than maximum, maximum should be constant Show more
right_tail(x, minimum = 0) genius	Combo, Regular	NaN everything less than minimum, minimum should be constant Show more
trade_when(x, y, z) base	Combo, Regular	The trade_when operator changes Alpha values only when a specific condition is met, keeps previous values otherwise, and can close positions by assigning NaN under an exit condition. It is useful for reducing turnover and controlling when trades are executed. Show more

Group

Operator	Scope	Description
combo_a(alpha, nlength = 250, mode = 'algo1') base	Combo	Combines multiple alpha signals into a single weighted output by balancing each alpha's historical return with its variability over the most recent nlength days. The parameter mode selects one of the several weighted approaches (algo1, algo2, algo3), each of which handles the tradeoff between performance and stability differently.
group_backfill(x, group, d, std = 4.0) base	Combo, Regular	Fills missing (NaN) values for instruments within the same group by calculating a winsorized mean of all non-NaN values over the past d days. The winsorized mean is computed by trimming extreme values based on a specified standard deviation multiplier (std, default 4.0). Show more
group_cartesian_product(g1, g2) genius	Combo, Regular	Merge two groups into one group. If originally there are len_1 and len_2 group indices in g1 and g2, there will be len_1 * len_2 indices in the new group.

weight, group) base		Show more
group_median(x, group) genius	Combo, Regular	All elements in group equals to the median value of the group.
group_neutralize(x, group) base	Combo, Regular	Neutralizes Alpha values within each specified group by subtracting the group mean from each value. Groups can be industry, sector, country, or any custom grouping. Show more
group_normalize(x, group, constantCheck=False, tolerance=0.01, scale=1) genius	Combo, Regular	Normalizes input such that each group's absolute sum is 1
group_rank(x, group) base	Combo, Regular	Ranks each element within its group based on the input field, assigning a value between 0.0 and 1.0. This helps compare items within the same group, such as stocks in the same industry. Show more
group_scale(x, group) base	Combo, Regular	Normalizes values within each group to a range between 0 and 1, making data comparable across different groups. Show more
group_zscore(x, group) base	Combo, Regular	Calculates the Z-score of each value within its group, showing how far each value is from the group mean in terms of standard deviations. Useful for comparing values relative to their group. Show more

Special

Operator	Scope	Description
in base	Selection	in
self_corr(input) base	Combo	Taking an input matrix of (D x N) with lookback="K", producing an output matrix of (D x N x N), where each output(di, j, k) refers to correlation of input(di-K:di, j) and input(di-K:di, k). Outputs (D x N x N) from the input of (D x N)
universe_size base	Selection	universe_size

Operator	Scope	Description
reduce_avg(input, threshold=0) base	Combo	Average of non-NAN elements of d(..., :). Threshold: Minimum required number of valid (non-nan) values. If there is not enough valid values, then the output is nan. 0 means no limit.threshold (Default: 0) *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_choose(input, nth, ignoreNan=true) base	Combo	Choose the 'nth' element in the array, return NAN if not found. Threshold: nth="" (Required) ignoreNan="true|false" (Default: true) *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_count(input, threshold) base	Combo	Count the number of element of d(..., :) > threshold. threshold= *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_ir(input) base	Combo	IR of values in the array *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_kurtosis(input) base	Combo	Kurtosis of values in the array ***Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. If input matrix is (D x N), output matrix (D x 1) If input matrix is (D x N X N), output matrix (D x N X 1) The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_max(input) base	Combo	Maximum of elements of d(..., :) *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_min(input) base	Combo	Minimum of elements of d(..., :) ***Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. If input matrix is (D x N), output matrix (D x 1) If input matrix is (D x N X N), output matrix (D x N X 1) The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_norm(input) base	Combo	Absolute sum of number of element of d(..., :) *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).

e(input, percentage=0.5) base		percentage=0.5. Threshold: percentage=*** (Default: 0.5) *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_powersum(input, constant=2, precise=false) base	Combo	Sum of power, sum(power(x, constant)). Threshold: precise, whether calculate power precise if constant greater than 4, default false constant=, default:2 *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_range(input) base	Combo	Return the range of values in the array, return NAN if no valid value *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_skewness(input) base	Combo	Skewness of values in the array *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_stddev(input, threshold=0) base	Combo	Standard deviation of values in the array. Threshold: Minimum required percentage of valid (non-nan) values. If there is not enough valid values, then the output is NAN. 0 means no limit.threshold (Default: 0) *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).
reduce_sum(input) base	Combo	Sum the number of element of d(..., :) *** Takes an input 2-D or 3-D matrix with user-defined reducer, producing an output matrix. *If input matrix is (D x N), output matrix (D x 1) *If input matrix is (D x N X N), output matrix (D x N X 1) *The defined function is applied on the last dimension : output(I) = reduce(input(I, 0:N)).